Programming Project #4: Routing

Computer Networks I CSCI-471 and CSCI 598

In this project you will get a chance to write the code emulates the behavior of a standard IPv4 router. You will be given two input files (1) configuration for the router in the form of a list of interfaces and associated subnets and (2) a route table in the form of a list of networks and the next-hop used to reach that subnet. After processing the configuration files your router will receive a series of packets and you must determine where the packet should be sent (the "packets" will really a list of IP addresses you need to process).

I'm hopeful this be a fun and interesting way to begin to visualize how routing and subnets work together to get your traffic where it needs to go. I'm not giving very much direction about how to implement the functional requirements, you may approach the design of this assignment however you like.

Functional Requirements (CSCI-471 and 598J)

At a minimum, your submission must do the following to receive full credit.

- **Basic requirements:**
  - The executable should be called **router**.
  - Your program must be written in C/C++, and must compile and run on Ubuntu 22.04LTS. We will test your submission in the virtual lab.
  - Your program should take two required command line arguments
    - -c <interface configuration file>
    - -r <route table>
  - Your program should take three optional command line arguments
    - -i <input file>. If the input file is not specified, read from stdin
    - -o <output file> If the output file is not specified, write to stdout
    - -d <debug level> If the debug level is not specified, set the level to 4.

- **File Formats**
  - Each of the files used for this assignment will be a multi-line text file.
  - Line termination will follow the *nix convention, a single linefeed character.
  - The contents of each line may have any number of whitespace characters before or after the line's contents.
  - The three input files may have comments in them. Comments are identified by a has (#) as the first non-whitespace character on the line. When reading input files you should ignore comment lines.
  - With this format you should be able to use the iostream library to read one line of the file at a time, then use the regular expression library to parse the fields.
  - **Interface Configuration File:**
    - The format for each line is:
      `InterfaceName<SP+>IPAddress/netmask`
    - For example the line **`eth0 138.67.1.2/18`** indicates an interface named eth0, assigned to IP address 138.67.1.2 with a netmask that is 18 bits long.
    - There may be any amount of whitespace (space or tab) between the InterfaceName and the IPAddress.
    - There will be no whitespace between the IP address and the netmask.

- **Route Table:**
  - The format for each line is
    `Network/netmask<SP+>nexthop`
  - For example the line **`192.168.12.0/24 138.67.1.4`** indicates a network with the **24 bit** prefix **192.168.12.0** can be reached through the router at **138.67.1.4**.
  - There may be any amount of whitespace (space or tab) between the Netmask and the next-hop IP address.
  - There will be no whitespace between the network address and the netmask.
- **Input File**
  - The format for each line is:
    `destinationIP`
  - For example, the line **138.67.20.1** indicates a packet that should be forwarded out the appropriate interface with the correct destination IP addess.
- **Output File**
  - There are two possible output formats. The first:
    destinationIP:<SP+>`InterfaceName<SP+>-><SP+> nextHop`
  - For example, line "138.67.20.1: eth4 -> 138.67.20.254" the indicates a packet destined for 138.67.20.1 should be forwarded out interface eth4 to the node with IP address "138.67.20.254"
  - If the destination IP is part of a subnet that is directly attached to the router, then the nextHop IP address should be the same as the destinationIP
  - The other output line format should be used if there destinationIP address is not reachable. Note this can only happen when there is no default route in the input file.
    destinationIP:<SP+>unreachable

- **Functionality**
  - For each line in the input file:
    - Read the interface configuration and the route table files. Building the appropriate data structures as you read the file.
    - Read one line from the input file. Apply the rules of subnets and routing to determine the correct next hop and find the interface that can be used to reach that next hop.
    - Print a correctly formatted line to the output file.
  - When you have read and processed all the lines from the input file, close the output file and exit.

Additional Constraints:

- We will not provide intentionally broken configurations.
  - In the routing table there will not be entries with next-hop addresses that are not directly attached to one of the local subnets.
  - Each interface will have a unique name and configuration. However, remember that one interface might be attached to a network that is a subnet of another network. For

example “eth0 138.67.20.3/22” and “eth1 138.67.20.254/24” would be a valid configuration.

- Grading will be done by comparing your output file to correct output, so please follow the output file format closely (some output files will be provided for comparison).
- You may not use any external libraries beyond the C/C++ standard libraries. Specifically, you should not use any library you might find that does operations on IP addresses or routing.

Hints and Suggestions.

There is no starter code, you may use the Makefile, command line argument parser and logging.h file provided with the first three assignments if you like. The supplemental document has an outline and some suggestions for class/functions you might find helpful. You are under no obligation to use those suggestions.

What to submit.

This assignment is due by the end of the day on Sunday December 7th. Because it is the end of the semester and we must have time to grade, no late submissions will be accepted without prior arrangement. Contact me if you are going to need an extension.

You should submit a tarball of a single directory containing a makefile, a README.txt with your name and any information I need to compile the program and the source files needed to build your program. The single directory must be named with your username. The grader should not need to do anything other than untar your files, cd into your directory, type make and begin testing. Do not include any core, object or binary files in the tarball.

In addition to functionality, you will be graded on the quality of the code, including readability, comments and the use of proper programing practices.